

Recherche et sauvetage autonomes

Projet B - IA712 Robotique mobile, projet final

Équipe **RobotZ**

ROS 2 Humble · Ignition Gazebo Fortress · TurtleBot 4 · Nav2 · slam_toolbox · BehaviorTree.CPP

Dépôt : <https://github.com/YiZeems/autonomous-search-and-rescue>

Membre	Courriel	Membre	Courriel
Julien GIMENEZ	julien.gimenez@telecom-paris.fr	Paul CINTRA	paul.cintra@telecom-paris.fr
Hugo FANCHINI	hugo.fanchini@telecom-paris.fr	Yimou ZHANG	yimou.zhang@telecom-paris.fr

Résumé. Un unique TurtleBot 4 explore de façon autonome une arène de catastrophe cloisonnée et inconnue, construit une carte par SLAM et localise quatre « victimes » (AprilTags) en projetant les détections caméra dans le repère global `map` à l'aide de `tf2`. La prise de décision repose sur un *Behavior Tree* (et non une FSM, comme exigé [1]) qui orchestre une **mission en deux phases** : exploration par frontières, puis inspection des pièces déduites de la carte. Le run autonome final atteint **97,35 % de couverture et 4/4 victimes** en un seul clic. Ce rapport présente l'architecture du système, les fondements théoriques, le cheminement décisionnel de l'équipe à travers trois runs représentatifs et les enseignements tirés.

1 Énoncé du problème et exigences

La robotique de recherche et sauvetage pose une question simple mais exigeante : lorsqu'un environnement est trop dangereux pour des humains - un bâtiment effondré, un sous-sol inondé, une zone d'accident nucléaire - un robot peut-il y entrer seul, comprendre un espace qu'il n'a jamais vu et en rapporter la seule information qui compte, à savoir *où se trouvent les victimes* ? Ce projet est une réponse simulée et autonome à cette question, et une synthèse de l'ensemble du projet IA712 : il relie SLAM, exploration autonome, navigation et prise de décision en un système unique qui se lance d'une seule commande et ne requiert aucune intervention humaine.

Le scénario [1] place un robot dans une zone de catastrophe simulée où les humains ne peuvent aller. Le robot doit *explorer de façon autonome* un environnement inconnu, *localiser les victimes* (AprilTags) et *rapporter leurs coordonnées* dans un repère global. Ce qui rend la tâche intéressante, c'est précisément que rien n'est connu à l'avance : le robot n'a pas de carte, il ignore combien de pièces existent et où se trouvent les victimes, et il doit décider *par lui-même* où aller ensuite, quand il en a vu assez et quand la mission est terminée. L'énoncé fixe les contraintes que nous satisfaisons :

Exigence [1]	Comment b1 la satisfait
Exploration autonome (frontières / RRT), <i>sans intervention humaine</i>	Exploration par frontières (Yamauchi [2]) avec sélection par gain d'information [3], cadencée par le BT
Qualité de la carte, fermeture de boucle	<code>slam_toolbox</code> async avec fermeture de boucle [4]
Détection caméra, projection vers <code>map</code> via <code>tf2</code>	<code>apriltag_ros</code> (tag36h11) [7] → <code>victim_registry</code> projette caméra→ <code>map</code> (TF2) [8]
Carte > 90 % de la zone	97,35 %
Carte finale annotée de toutes les positions des victimes	carte annotée + 4 victimes, Fig. 10
Décision = Behavior Tree, <i>pas</i> de FSM	BehaviorTree.CPP v3 [6]
Lancement en un clic	<code>ros2 launch rescue_bringup bringup_tb4.launch.py</code>
Bonus : couverture glouton vs. gain d'information	benchmark quantitatif ($n=3$), §3

Le système en un coup d'œil.

L'arène est un bâtiment cloisonné de quatre pièces (`rescue_arena`) avec une AprilTag « victime » sur un mur extérieur de chaque pièce. Le robot est un **TurtleBot 4** (base Create 3, RPLIDAR A1M8, caméra OAK-D) simulé sous **Ignition Gazebo Fortress**. À partir d'un seul lancement, le robot cartographie le bâtiment par SLAM, visite de façon autonome chaque pièce, détecte les quatre tags, les projette dans le repère `map` et s'arrête sur une carte annotée et propre - *sans* connaissance préalable des pièces ni des positions des victimes. Les sections suivantes décrivent l'architecture, les concepts qu'elle applique, et comment nous y sommes parvenus.

2 Architecture du système

L'espace de travail est découpé en **quatre paquets ROS 2** afin que les quatre membres puissent travailler en parallèle : **rescue_bringup** (lancement + configs), **rescue_robot** (« mégapaquet » Python : exploration, perception, navigation/inspection, résultats), **rescue_world** (mondes Ignition + cibles AprilTag) et **rescue_decision** (le Behavior Tree en C++). La Fig. 1 est le schéma système détaillé exigé par [1].

Ce découpage est délibéré. Une séparation nette entre *ce que le robot perçoit* (SLAM, perception), *ce qu'il fait* (exploration, navigation, inspection), *où il agit* (le monde simulé) et *comment il décide* (le Behavior Tree) a permis à quatre personnes de travailler en parallèle sur leurs propres branches sans se gêner, et a maintenu chaque composant testable isolément - nous pouvions exercer le Behavior Tree, le registre des victimes et la visualisation RViz contre de légers publieurs factices, bien avant que la pile Gazebo complète n'existe. Tout aussi important, la conception trace une frontière nette entre *politique* et *mécanisme* : le Behavior Tree décide *quelle* phase doit s'exécuter et *quand* passer à la suivante, tandis que les nœuds Python sont de purs exécutants qui ignorent tout de la mission globale. C'est cette frontière qui rend le système facile à raisonner, et c'est ce que déploient les deux sections suivantes : d'abord les concepts sur lesquels reposent les mécanismes, puis la mission que la politique assemble à partir d'eux.

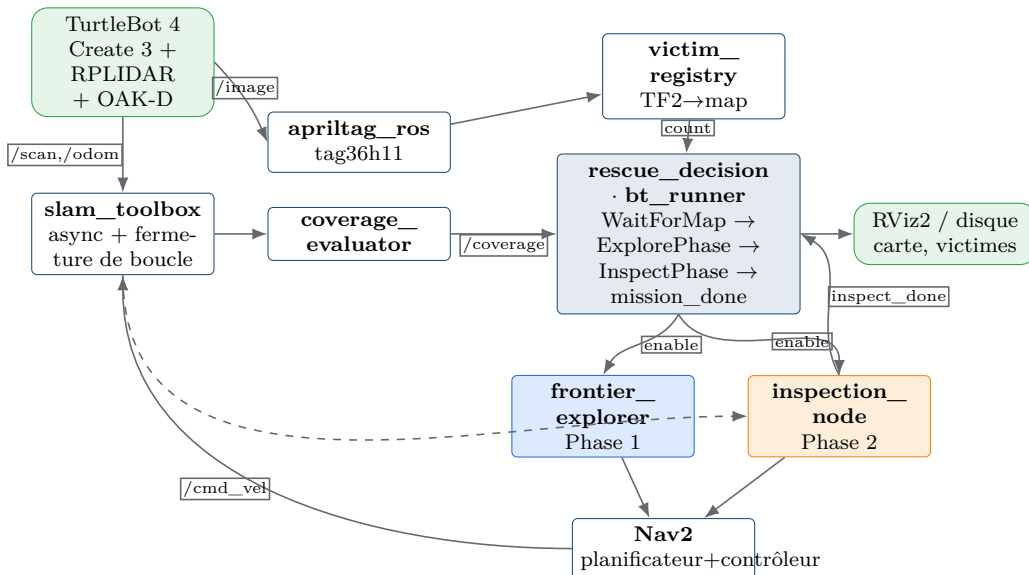


Figure 1: Architecture du système et flux de topics. Le Behavior Tree (**rescue_decision**) est l'*orchestrateur* : il cadence l'explorateur de la Phase 1 et l'inspecteur de la Phase 2 via les topics `/mission/*` ; les nœuds Python font le travail. Le SLAM alimente la carte vers l'exploration, la couverture, Nav2 et l'inspection.

Les paquets et leurs responsabilités :

- **rescue_bringup** - le point d'entrée *en un clic* (`bringup_tb4.launch.py`) et toutes les configs (Nav2, SLAM, AprilTag). Un seul lancement démarre la simulation, le SLAM, Nav2, la perception, l'explorateur/inspecteur et le Behavior Tree.
- **rescue_robot** - le « mégapaquet » Python : **exploration/** (recherche de frontières + liste noire), **navigation/** (planificateur d'inspection + nœud), **detection/** (registre/détecteur de victimes), **results/** (évaluateur de couverture + exporteur de résultats) et des mocks de développement.
- **rescue_world** - l'arène Ignition Fortress (`rescue_arena.sdf`) et les cibles AprilTag en voxels. Une *source unique* possède le placement des victimes, de sorte que la carte et la vérité terrain ne divergent jamais.
- **rescue_decision** - le véritable exécuteur C++ `BehaviorTree.CPP v3` (visualisable dans Groot), avec le XML de l'arbre de mission. C'est l'exigence « décision = BT, pas de FSM ».

Pipeline de perception.

Les victimes sont des AprilTags `tag36h11` (l'énoncé exclut explicitement toute détection humaine sophistiquée / YOLO). Une difficulté subtile : un tag à texture PBR se rend *blanc* sous Ogre2+Mesa, de sorte que `apriltag_ros` ne voit rien ; nous construisons chaque tag en **boîtes de voxels** avec

un anneau blanc de zone calme autour du marqueur natif 8×8 (un redimensionnement naïf en 10×10 détruit la zone calme et rend le tag *affiché mais indétectable*). Chaque détection fournit une transformée `camera`→`victim_i` ; `victim_registry` la projette vers `map` via TF2, déduplique par ID et persiste `results/victims.json` - l'exigence « rapporter leurs coordonnées dans le repère global ».

3 Fondements théoriques

SLAM et fermeture de boucle.

Le robot construit la carte en ligne avec `slam_toolbox` (SLAM par graphe de poses). La fermeture de boucle corrige la dérive accumulée lorsque le robot ré-observe un lieu connu, gardant la carte globale cohérente lors de passages répétés [4,9].

Exploration par frontières et gain d'information.

Une *frontière* est la limite entre les cellules libres et inconnues [2]. La sélection gloutonne prend la frontière la plus proche ; la sélection par *gain d'information* maximise $g(f) = \text{gain}(f) - \lambda \cdot \text{cost}(f)$ [3], où `cost` est la *longueur du chemin Nav2* (pas euclidienne). Notre benchmark $n=3$ confirme la théorie : le gain d'information atteint $0,85 \pm 0,08$ de couverture (la seule au-dessus de 90 %) contre 0,72 pour le glouton. *Pourquoi* l'écart se creuse *dans le temps* (Fig. 3) : le glouton, attiré par la frontière la plus proche, raffine la zone déjà ouverte et repousse les pièces lointaines, donc plafonne ($\sim 67-70\%$) ; le gain d'information préfère la frontière qui révèle le plus d'inconnu *par mètre* (une porte ouvrant une pièce entière), donc chaque déplacement rapporte plus et la courbe franchit les 90 % - et comme le coût est la longueur de chemin Nav2 (pas euclidienne), les murs sont pris en compte. Un ajout clé de robustesse est une *liste noire de frontières inaccessibles* indexée par une coordonnée monde quantifiée, de sorte qu'une frontière morte ne soit jamais resélectionnée même lorsque l'origine de la grille se décale. Concrètement, une frontière est blacklistée soit quand Nav2 échoue à l'atteindre, soit quand elle est re-sélectionnée **3 fois de suite sans gain de couverture** (le cas courant dans notre arène) ; elle n'est alors plus jamais reproposée.

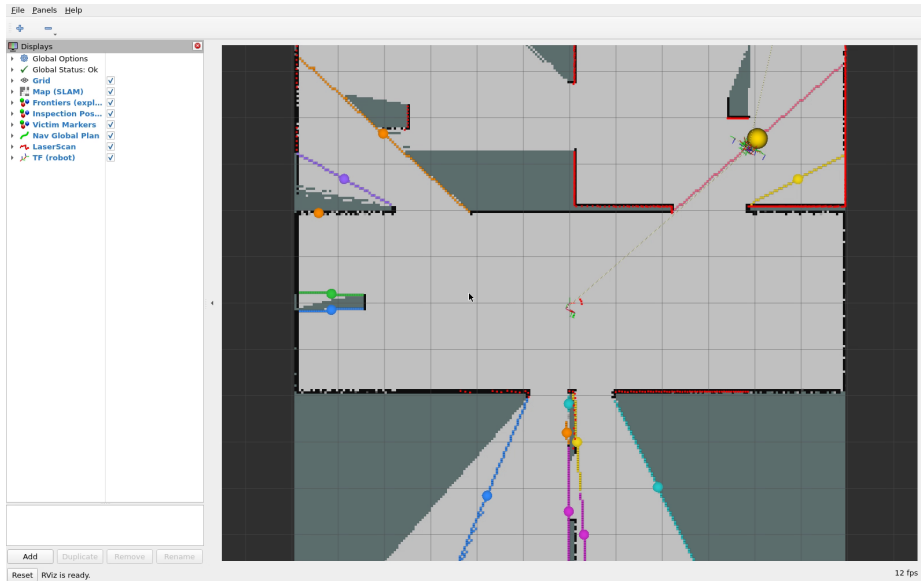
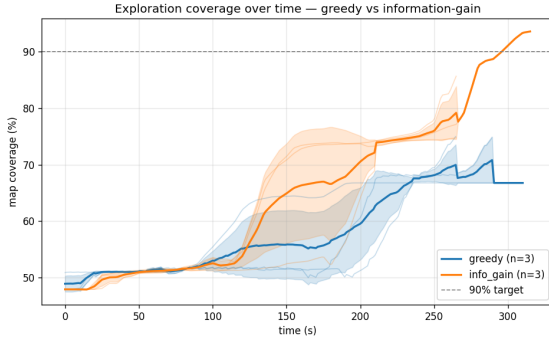


Figure 2: Exploration par frontières dans RViz, forme et couleur de chaque élément : **cubes colorés** = cellules de frontière (limite connu/inconnu), **une couleur par cluster** (composante connexe) ; **sphères de même couleur** = centroïdes des clusters ; **grosse sphère jaune** = frontière-but retenue ($g = \text{gain} - \lambda \text{cost}$) ; **ligne bleue** = plan Nav2 ; **axes RVB** = robot (TF). Fond clair = libre, gris-bleu = inconnu, traits noirs/rouges = murs.

Planification de chemin et contrôle.

Le planificateur global produit un chemin géométrique ; le contrôleur local le suit. Nous avons comparé *Dijkstra/NavFn* vs. *A* (Smac)* [9] : dans une arène fortement cloisonnée, le robot se tient souvent à l'intérieur de la couche d'inflation, où *A** refuse de planifier (« start in lethal space ») ; NavFn le tolère et est conservé. Le contrôleur est DWB (par échantillonnage).

Repères de coordonnées et TF2.



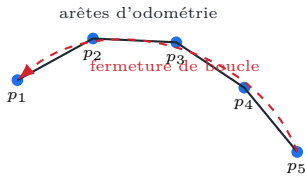
(a) Bonus ($n=3$) - couverture cartographique *au fil du temps* : le **gain d'information** (orange) atteint la cible 90 %, le **glouton** (bleu) plafonne ~67–70 %.

Figure 3: Stratégie d'exploration (bonus) et comparaison des planificateurs globaux.

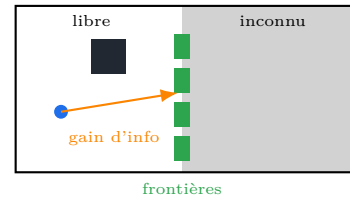
Une victime est détectée dans le repère *camera* ; *tf2* enchaîne *camera* → *victim_i* → *map* pour rapporter une position globale (Fig. 5b), exactement l'exigence « projeter les positions dans le repère global » [1,8].

Behavior Trees vs. FSM.

L'énoncé interdit les FSM. Un BT est réactif et composable : une **Sequence** avec mémoire tique ses enfants de gauche à droite, *reprend* au nœud RUNNING et n'avance que sur SUCCESS - une véritable séquence de phases plutôt que des transitions d'état codées à la main [6].

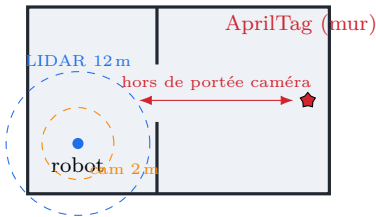


(a) Le SLAM est un graphe de poses : ré-observer un lieu connu ajoute une arête de *fermeture de boucle* qui corrige la dérive accumulée et garde la carte globale cohérente sur des passages répétés.

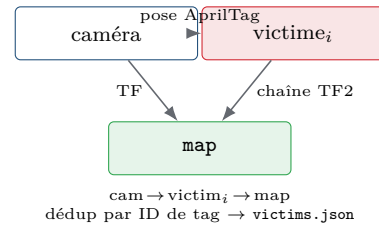


(b) Une *frontière* est la limite libre/inconnu (vert) ; le gain d'information sélectionne la plus informative, notée par le coût de *chemin* Nav2 (pas euclidien).

Figure 4: Deux concepts clés : SLAM par graphe de poses avec fermeture de boucle et exploration par frontières avec gain d'information.



(a) Écart caméra-LIDAR : le LIDAR cartographie une pièce depuis l'embrasure, mais la caméra n'approche jamais à 2m du tag mural ⇒ l'exploration pure manque les victimes.



(b) Projection TF2 d'une détection dans le repère global *map*.

Figure 5: Deux concepts clés derrière la mission.

La mission en deux phases.

L'exploration pure complète la *carte* mais plafonne la détection des victimes (Fig. 5a). Le remède est une seconde phase *autonome* : à partir de la carte qu'il vient de construire, **inspection_node** dérive une *pose par pièce découverte* (face au mur le plus proche, alignée sur une cellule navigable, *sans coordonnées de victimes*) ; le robot s'y rend et balaie la caméra. L'arbre de mission BT est la **Sequence** WaitForMap → ExplorePhase → InspectPhase → VictimsFound → mission_done (Fig. 6).

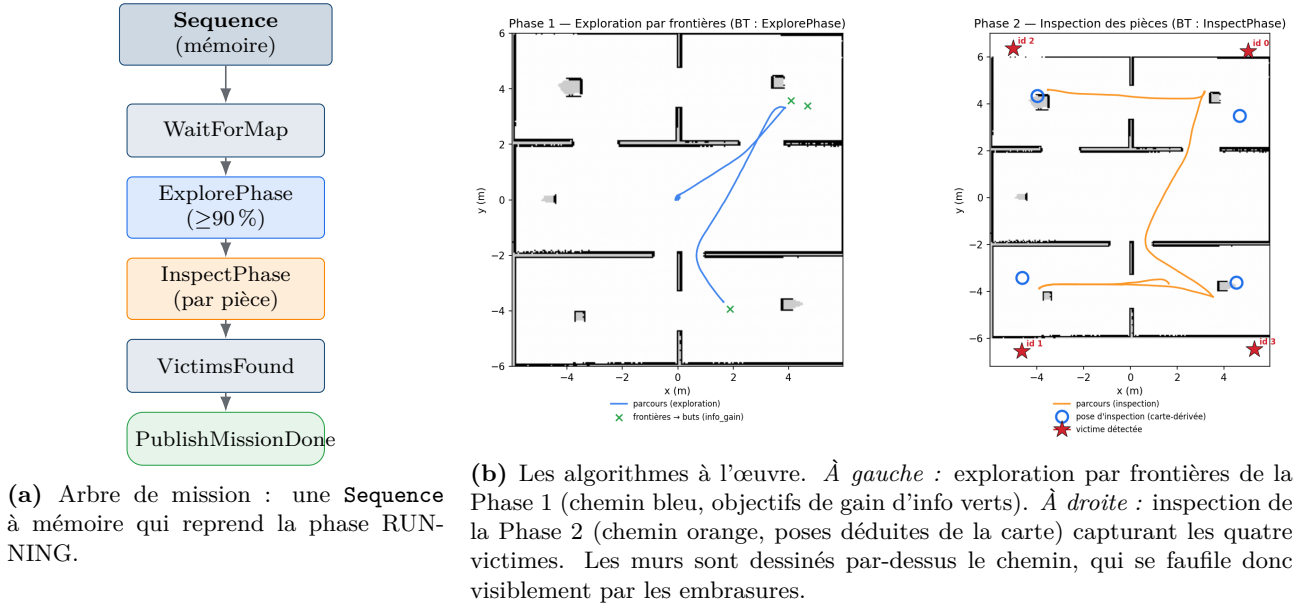


Figure 6: La mission en deux phases orchestrée par le Behavior Tree.

Concepts appliqués dans le projet.

Le Tableau 1 associe chaque concept à l'endroit où il est appliqué dans notre système.

Concept	Où il est appliqué dans b1
SLAM + fermeture de boucle	<code>slam_toolbox</code> (async) : <code>/map</code> en ligne + TF <code>map</code> → <code>odom</code> , fermeture de boucle sur les revisites
Exploration par frontières	<code>frontier_explorer_node</code> : limite libre/inconnu, liste noire indexée monde
Gain d'information (bonus)	sélection de frontières par gain d'info ; benchmark quantitatif vs. glouton ($n=3$)
Planification de chemin	planificateur global Nav2 - NavFn (Dijkstra) conservé ; A* (Smac) comparé
Contrôle local	contrôleur Nav2 DWB (scoring de trajectoires par échantillonnage)
Repères de coordonnées / TF2	<code>victim_registry</code> : projection caméra→ <code>victim_i</code> → <code>map</code>
Behavior Trees (décision)	<code>rescue_decision</code> (BT.CPP v3) : orchestration de la mission en deux phases

Table 1: Les concepts clés, et exactement où chacun est appliqué dans le projet.

4 Cheminement décisionnel : trois runs représentatifs

Les chiffres phares - 97 % de couverture, quatre victimes, un clic - masquent une route longue et instructive. Rien n'est venu d'un coup : presque chaque capacité que nous tenons aujourd'hui pour acquise a été, à un moment, un run qui faisait la mauvaise chose pour une raison que nous ne comprenions pas encore. Nous avons tenu un journal décisionnel écrit au format *Problème* → *Investigation* → *Décision* → *Pourquoi*, et l'habitude la plus utile qu'il nous a apprise a été de *mesurer avant de prescrire* - de laisser un journal d'événements ou une métrique, plutôt qu'une intuition, nous dire ce qui n'allait réellement pas. Trois runs distillent cette histoire ; chacun est rapporté ci-dessous comme ce qui a *bloqué* et ce que nous avons *amélioré*, et la Fig. 7 en est la synthèse sur une page.

Run 1 - exploration autonome pure (v20-v22).

Notre premier système complet faisait la chose évidente : exploration par frontières pilotant Nav2, avec `slam_toolbox` construisant la carte en dessous. Cela marchait, au sens où il cartographiait de façon autonome environ quatre-vingt-dix pour cent du bâtiment - mais il ne trouvait toujours qu'environ deux des quatre victimes, et pendant longtemps nous ne voyions pas pourquoi. La réponse s'est révélée géométrique plutôt qu'algorithmique : un LIDAR de douze mètres cartographie confortablement une pièce depuis son embrasure, donc un robot qui optimise la *couverture* n'a aucune raison de s'approcher d'un mur, alors que la caméra ne reconnaît un tag qu'à deux mètres environ. L'exploration était donc *complète* et la détection *structurellement incomplète* en même temps. En chemin, ce run nous a aussi enseigné deux leçons tranchantes - une frontière que le planificateur ne peut jamais atteindre piégera le robot dans une boucle infinie tant qu'elle n'est pas mise en liste noire,

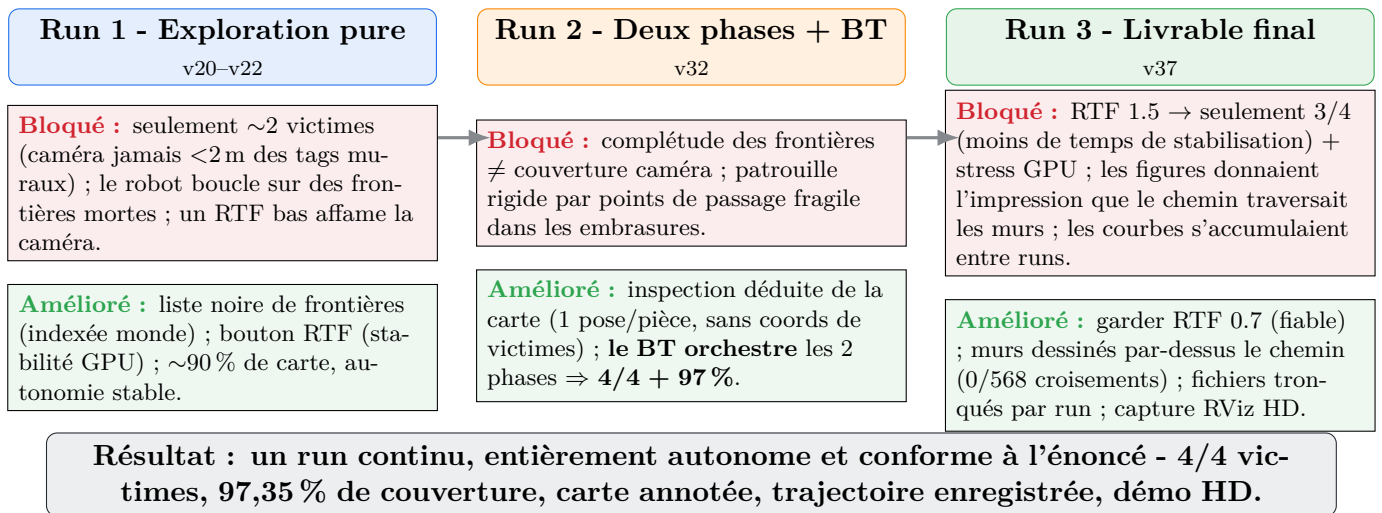


Figure 7: Le cheminement décisionnel sur une page : trois runs, chacun comme *ce qui a bloqué* (rouge) → *ce que nous avons amélioré* (vert). Deux causes racines distinctes furent démasquées tard - la fonction de coût Nav2 décourageait l'entrée dans les pièces (couverture corrigée), et un RTF effondré affamait le renderer caméra (détection corrigée) ; le Behavior Tree à deux phases a ensuite rendu le 4/4 fiable.

et un facteur temps réel assez bas pour affamer le renderer GPU de la caméra arrêtera silencieusement la détection alors que les métadonnées de la caméra continuent de circuler et masquent le défaut. Nous avons quitté ce run avec une carte propre, une autonomie fiable et un diagnostic clair de ce qui manquait encore.

Run 2 - deux phases, orchestré par BT (v32).

Le correctif a découlé directement de ce diagnostic : si le robot ne s'approche pas des murs en explorant, donnons-lui une seconde raison de le faire. Nous avons ajouté une phase d'*inspection* qui lit la carte fraîchement construite, dérive une pose par pièce découverte à environ 1,3m du mur le plus proche - dans la portée de la caméra, mais en utilisant uniquement la carte et jamais les positions des victimes - et visite chacune tour à tour en balayant la caméra. Surtout, nous avons aussi promu le Behavior Tree d'un superviseur passif qui se contentait de surveiller la métrique de couverture en véritable *orchestrateur* de la mission, cadencant exploration et inspection en séquence ; c'est ce qui rend le système conforme à l'exigence « décision par Behavior Tree, pas de FSM » sur le fond et pas seulement sur le nom. Le résultat fut la percée que nous cherchions : **quatre victimes sur quatre et 97 % de couverture** en un seul run continu et entièrement autonome.

Run 3 - le livrable final.

La mission fonctionnant, le dernier run a consisté à transformer un résultat en un livrable fiable et reproductible. Nous nous sommes d'abord demandé si nous pouvions simplement faire tourner la simulation plus vite : pousser le facteur temps réel à 1,5 finissait bien plus tôt, mais laissait moins de temps à la caméra pour se stabiliser à chaque pose et nous ramenait discrètement à trois victimes sur quatre, en plus de stresser le GPU - nous avons donc gardé le facteur plus lent et fiable, petit rappel que pour un livrable, la répétabilité l'emporte sur la vitesse. Nous avons ensuite réglé un ensemble de problèmes de présentation qui, sans affecter le comportement du robot, affectaient la capacité d'un lecteur à *faire confiance* aux figures : la trajectoire dans le repère carte est désormais tracée avec les murs peints *par-dessus*, de sorte que le chemin n'apparaît jamais que dans l'espace libre et se faufile visiblement par chaque embrasure (nous avons vérifié qu'aucun de ses 568 segments ne traverse un mur) ; la carte d'occupation est rendue avec la bonne orientation verticale, ce qui était la vraie raison pour laquelle une figure antérieure *donnait l'impression* que le robot traversait les murs ; et les fichiers de résultats de chaque run sont tronqués au démarrage, si bien que les courbes d'un run ne s'empilent plus sur celles d'un autre. Le travail de robustesse qui permet à l'inspection de récupérer une pose inatteignable sur la machine d'un coéquipier (§3) a aussi atterri ici. Le résultat est le run sur lequel s'appuie tout ce rapport : **quatre victimes sur quatre, environ 97 % de couverture**, une carte annotée propre, une trajectoire enregistrée et une capture HD des algorithmes RViz en direct - une source unique et reproductible pour chaque figure présentée.

Difficultés et solutions en détail

Atteindre le 4/4 dans les Runs 1–2 a signifié éplucher six causes *empilées* - chaque correctif révélait le suivant (Tableau 2). L'intuition décisive : *deux* causes racines indépendantes se cachaient sous les symptômes - la fonction de coût Nav2 décourageait le robot d'*entrer* dans les pièces (un problème de **couverture**, corrigé en abaissant le rayon d'inflation et l'échelle de coût), et un facteur temps réel effondré *affamait le renderer caméra* alors que **camera_info** continuait de circuler (un problème de **détection**, très trompeur). Le Behavior Tree à deux phases a ensuite rendu le résultat fiable.

#	Symptôme	Cause racine	Décision (correctif)
1	Robot immobile, 0 victime	Points de patrouille en espace <i>inconnu</i>	Hybride : explorer d'abord, <i>puis</i> agir
2	Points rejetés/bloquants	Les objectifs tombent sur un mur	Valider les objectifs en espace libre
3	« Failed to make progress »	Réflexe de sécurité du Create 3	safety_override=full
4	Coins profonds rejetés à 93 %	Cellules de coin encore inconnues	Recalage sur la cellule libre la plus proche
5	Routage inter-pièces fragile	Pas de chemin global au moment de l'objectif	Pivot : rotation-balayage 360°
6	1 seule victime détectée	RTF bas affame le renderer caméra	Augmenter le RTF ; caméra coupée en exploration

Table 2: Six causes empilées derrière « 0 victime », et la décision prise à chaque étape.

Stabilité de l'hôte (WSL/GPU) - mesurer avant de prescrire.

Les longs runs redémarraient l'hôte Windows. Les coupables « évidents » étaient faux *deux fois* : la mémoire allait bien (**free** : 5/19 GiB) et aucune mise à jour Windows n'était en attente. Le journal d'événements a révélé les vraies causes : des bugchecks GPU VIDEO_TDR_FAILURE / CLOCK_WATCHDOG sous charge soutenue. Notre correctif contrôlable est le **bouton de facteur temps réel** (IA712_GZ_RT_RATE) : rester sur le GPU (le GL logiciel est $\sim 23\times$ plus lent) mais abaisser la cadence physique/rendu réduit la charge soutenue sans changer le temps *simulé* - donc la couverture et le `time_to_90` ne sont pas affectés.

5 Collaboration d'équipe et répartition des tâches

Nous avons travaillé sur des branches `dev/*` par membre, intégrées dans `dev/b1` (Fig. 8) ; `b1` est la version fusionnée que décrit ce rapport.

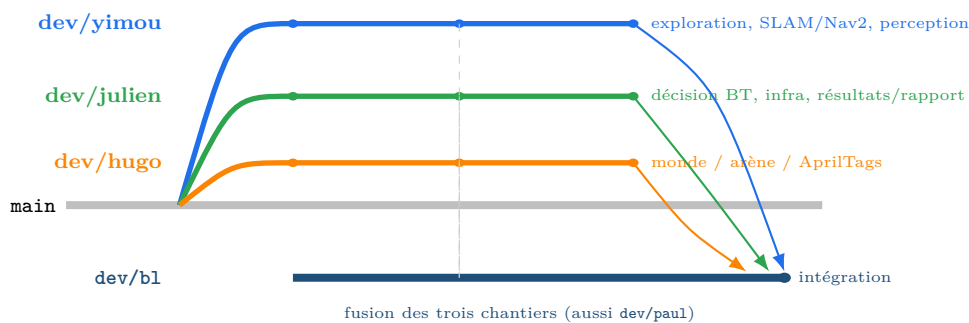


Figure 8: Construction des branches : trois branches de fonctionnalités par développeur (chantiers issus de l'historique git) sont fusionnées dans `dev/b1`, la branche d'intégration.

Répartition des tâches (d'après l'historique git) : **Yimou ZHANG** - exploration autonome, câblage SLAM/Nav2, cœur perception, benchmark ; **Julien GIMENEZ** - décision Behavior-Tree & orchestration en deux phases, infrastructure de simulation (WSL/GPU/RTF/DDS), résultats, intégration (`b1`) et rapport ; **Hugo FANCHINI** - monde de simulation, arène de sauvetage et placement des cibles AprilTag (`rescue_world`) ; **Paul CINTRA** - support et tests perception/détection (`dev/paul`).

6 Résultats

La Fig. 9 montre les algorithmes RViz en direct au fil de la mission ; la Fig. 10 les artefacts livrables. Tous proviennent de l'unique run final (`results/examples/118_nominal`). Une démonstration vidéo de la mission complète est disponible : <https://www.youtube.com/watch?v=BKXprWFloFs>.

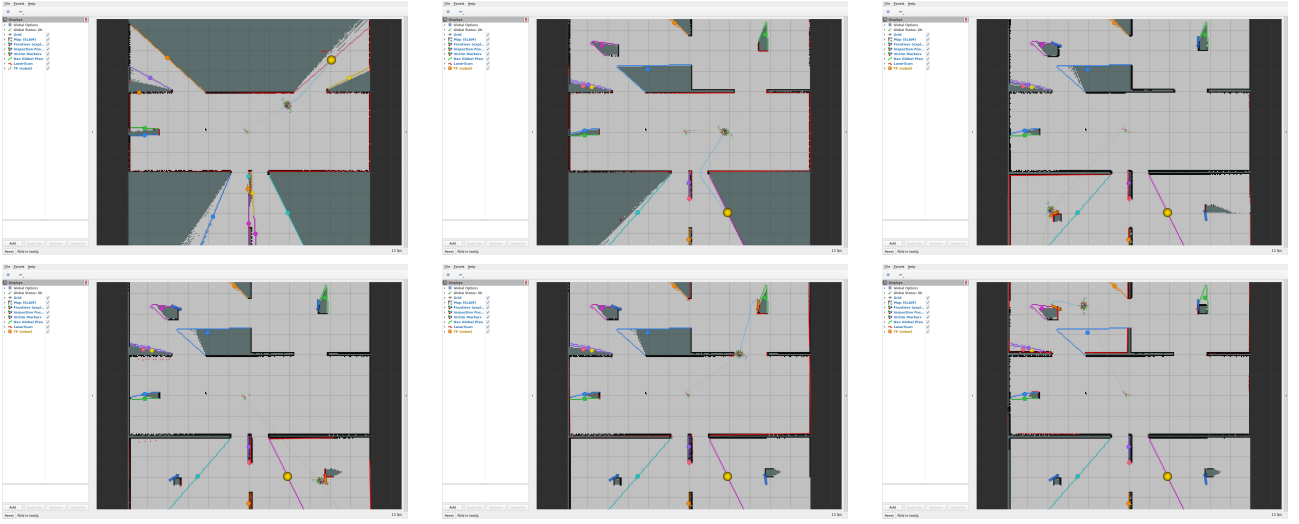


Figure 9: RViz en direct à six moments clés (1-3 : exploration précoce, carte presque construite, transition de phase ; 4-6 : inspection des pièces, victimes apparaissant, carte finale) - montrant la carte (SLAM), les frontières, les poses d'inspection, le plan Nav2, le LaserScan et les quatre marqueurs de victimes. Capturé en HD sur un serveur X virtuel.

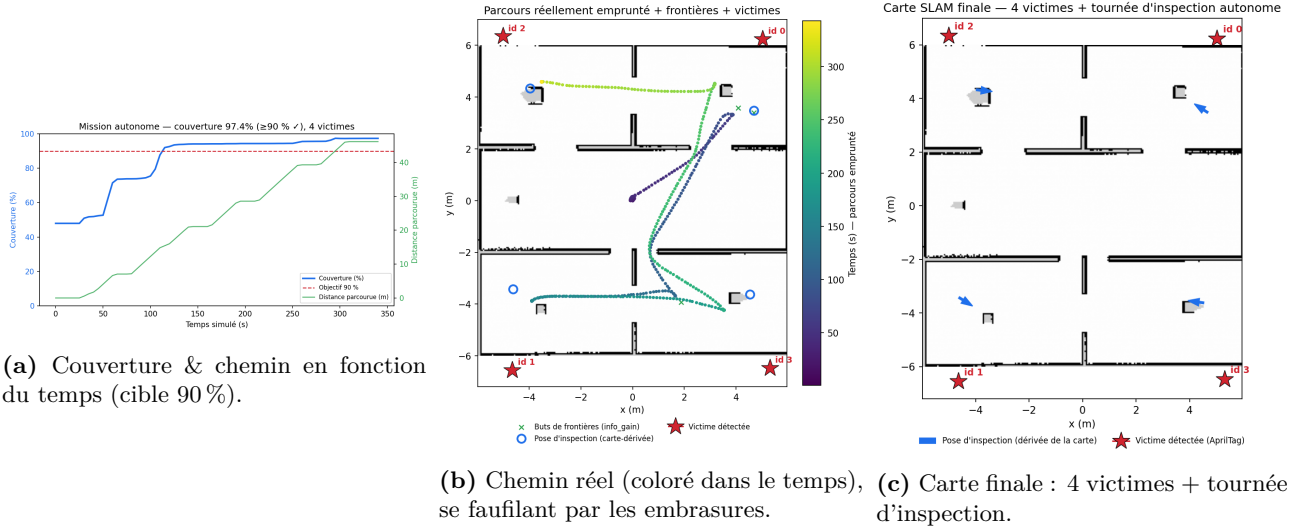


Figure 10: Résultats quantitatifs : **97,35%** de couverture et **4/4** victimes, entièrement autonome. La trajectoire est le chemin réel dans le repère carte ; les murs sont dessinés par-dessus, si bien qu'elle passe visiblement par les embrasures (vérifié : aucun segment ne traverse un mur).

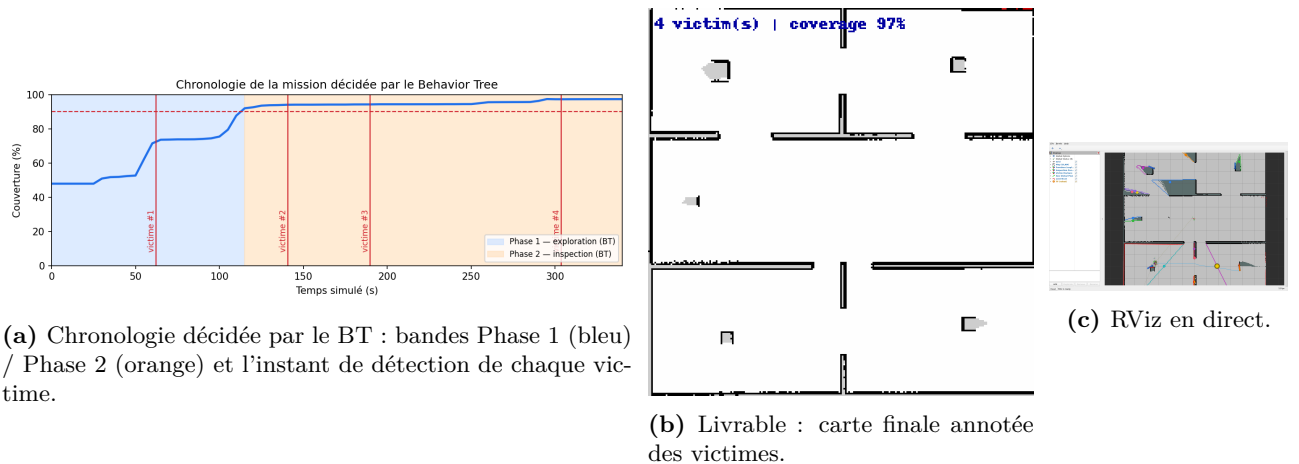


Figure 11: Chronologie de la mission et le livrable de l'énoncé (« carte finale annotée de toutes les positions des victimes »). La séparation des phases et les instants de détection confirment l'orchestration par le BT.

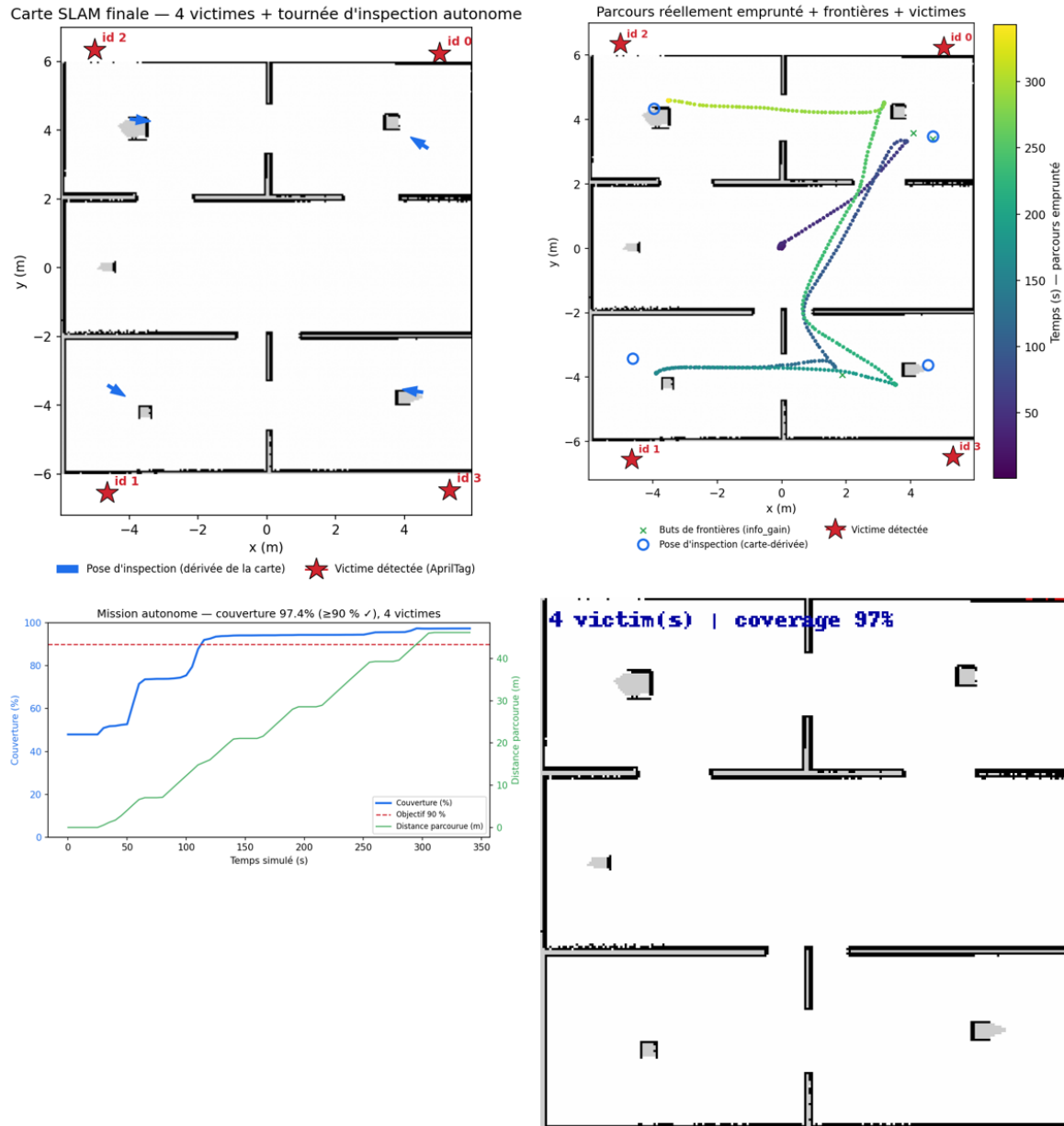


Figure 12: Montage de synthèse en un coup d’œil : carte finale avec les quatre victimes et la tournée d’inspection (en haut à gauche), la trajectoire réelle colorée dans le temps (en haut à droite), la couverture et la longueur du chemin au fil du temps (en bas à gauche) et la carte annotée livrable (en bas à droite). Tout est régénéré à partir de l’unique run archivé, de sorte que les figures sont reproductibles à partir des données.

Lecture des résultats.

La courbe de couverture (Fig. 10a) montre clairement les deux phases : une montée rapide pendant l’exploration jusqu’à la cible de 90 %, puis un plateau pendant l’inspection tandis que la longueur du chemin continue de croître (le robot se déplace encore, de pièce en pièce, mais la carte est déjà complète). La chronologie de mission (Fig. 11a) superpose les quatre instants de détection des victimes sur les bandes de phase : les premières victimes sont attrapées *pendant* l’exploration lorsque le robot passe par hasard près d’un tag mural, et les restantes *pendant* l’inspection - exactement le comportement que prédit la conception en deux phases. La trajectoire (Fig. 10b) confirme que le robot visite les quatre pièces et revient, en se faufilant par chaque embrasure.

Exécution et reproductibilité.

Toute la pile démarre d’une seule commande, `ros2 launch rescue_bringup bringup_tb4.launch.py`. Le run nominal est archivé (`results/examples/l18_nominal`) de sorte que chaque figure de ce rapport soit régénérable à partir des données ; le benchmark bonus est *reprenable* (chaque run persiste son résumé, les runs valides sont sautés au redémarrage), ce qui a permis à la campagne de survivre aux redémarrages de l’hôte. RViz est capturé en vidéo HD sur une machine sans écran via un serveur X virtuel, donnant la séquence d’algorithmes en direct de la Fig. 9.

7 Enseignements tirés et calendrier

Enseignements tirés.

Quatre leçons ressortent, et elles tiennent autant de la méthode d’ingénierie que de la robotique. La première est de *mesurer avant de prescrire*. Deux fois nous étions certains de savoir pourquoi la machine hôte redémarrait sans cesse - ce devait être la mémoire, puis sûrement une mise à jour Windows en attente - et deux fois nous avions tort ; seuls le journal d’événements, `free` et `nvidia-smi` ont désigné les vrais coupables, un plantage thermique du pilote GPU et, séparément, un `render` caméra affamé par un facteur temps réel effondré alors que ses métadonnées continuaient de circuler et masquaient le problème. La deuxième leçon est qu’un *correctif révèle le suivant* : passer de zéro à quatre victimes a signifié éplucher six causes empilées - un objectif non cartographié, un objectif sur un mur, un réflexe de sécurité, un coin inatteignable, un routage inter-pièces fragile et enfin la caméra affamée - où chaque correction ne faisait qu’exposer celle d’en dessous. La troisième est que *le robuste l’emporte sur le rigide* : dans une arène cloisonnée, un explorateur adaptatif qui visite déjà chaque pièce, suivi d’un balayage caméra complet à 360°, bat systématiquement une patrouille par points de passage écrite à la main qui se casse dès que la carte diffère légèrement. Et la quatrième est la *conformité par conception* - en faisant véritablement *orchestrer* la mission par le Behavior Tree plutôt que de la simplement surveiller, nous satisfaisons l’exigence « décision par BT, pas de FSM » de l’énoncé dans l’esprit et pas seulement sur le papier.

Calendrier (L13–L18).

Le projet a suivi le calendrier suggéré (Fig. 13).



Figure 13: Les six séances L13–L18 et leurs jalons.

8 Discussion et conclusion

Avec le recul, le projet réussit moins grâce à un quelconque algorithme astucieux que grâce à la manière dont les pièces sont amenées à coopérer. L’exploration est une recherche de frontières de manuel, le planificateur et le contrôleur sont des plugins Nav2 standard, le SLAM est `slam_toolbox` prêt à l’emploi, et le détecteur est `apriltag_ros` ; ce que nous avons apporté, c’est l’*intégration* - un Behavior Tree qui transforme ces composants indépendants en une mission cohérente, et une seconde phase autonome qui comble l’écart entre « la carte est complète » et « chaque victime est trouvée ». C’est exactement le glissement que demande l’énoncé : passer de l’écriture d’un code qui se contente de tourner à l’ingénierie d’un système dont les parties tiennent ensemble en conditions réelles.

La conception en deux phases est le cœur conceptuel du résultat. Un robot qui se contente d'explorer va, dans un bâtiment cloisonné, cartographier confortablement chaque pièce depuis son embrasure avec un LIDAR de douze mètres et n'aura jamais besoin d'y entrer - ce qui est efficace pour la cartographie mais fatal pour une caméra qui ne voit un tag qu'à deux mètres. Séparer la *cartographie* de l'*inspection*, et déduire les poses d'inspection de la carte que le robot vient de construire plutôt que d'une connaissance préalable des victimes, garde le système entièrement autonome tout en rendant la détection fiable. Le Behavior Tree est ce qui rend cette séparation nette : chaque phase est un nœud qui s'exécute jusqu'à sa propre condition de succès, et la mission se lit simplement comme une séquence.

La robustesse, enfin, fut une leçon en soi. Un run qui marche sur une machine n'est pas la même chose qu'un système qui marche : un costmap légèrement différent sur l'ordinateur d'un coéquipier suffisait à rendre deux poses d'inspection inatteignables, jusqu'à ce que l'inspecteur apprenne à attendre que le costmap se stabilise et à se rabattre sur une pose tirée vers l'intérieur de la pièce. La conclusion honnête de ce projet porte donc autant sur la méthode - mesurer, isoler une cause à la fois, rendre le système tolérant plutôt que rigide - que sur les chiffres finaux. Les prochaines étapes naturelles seraient de comparer A* à NavFn sur un costmap moins agressif, d'essayer un contrôleur Regulated Pure Pursuit pour des traversées d'embrasures plus douces, et d'ajouter un second robot en laissant le Behavior Tree coordonner une carte partagée.

Remerciements

Nous remercions chaleureusement **Zhi Yan**, enseignant-chercheur à l'ENSTA, pour le module IA712 et l'encadrement qui ont rendu ce projet possible.

Références

- [1] IA712 Mobile Robotics - Final Project, Project B (énoncé). Site : <https://yzrobot.github.io/IA712/>.
- [2] B. Yamauchi, *A frontier-based approach for autonomous exploration*, IEEE CIRA, 1997.
- [3] C. Stachniss et al., *Information gain-based exploration using Rao-Blackwellized particle filters*, ICRA, 2005.
- [4] S. Macenski, I. Jambrecic, *slam_toolbox: SLAM for the dynamic world*, JOSS, 2021.
- [5] S. Macenski et al., *The Marathon 2: A navigation system (Nav2)*, IROS, 2020.
- [6] D. Faconti et al., *BehaviorTree.CPP* library, <https://www.behaviortree.dev/>.
- [7] E. Olson, *AprilTag: A robust and flexible visual fiducial system*, ICRA, 2011; `apriltag_ros`.
- [8] T. Foote, *tf2: The transform library*, IEEE TePRA, 2013.
- [9] IA712 Mobile Robotics - supports de référence, <https://yzrobot.github.io/IA712/>.
- [10] Project repository, <https://github.com/YiZeems/autonomous-search-and-rescue>.